

COMPSYS302

Design: Software Practice

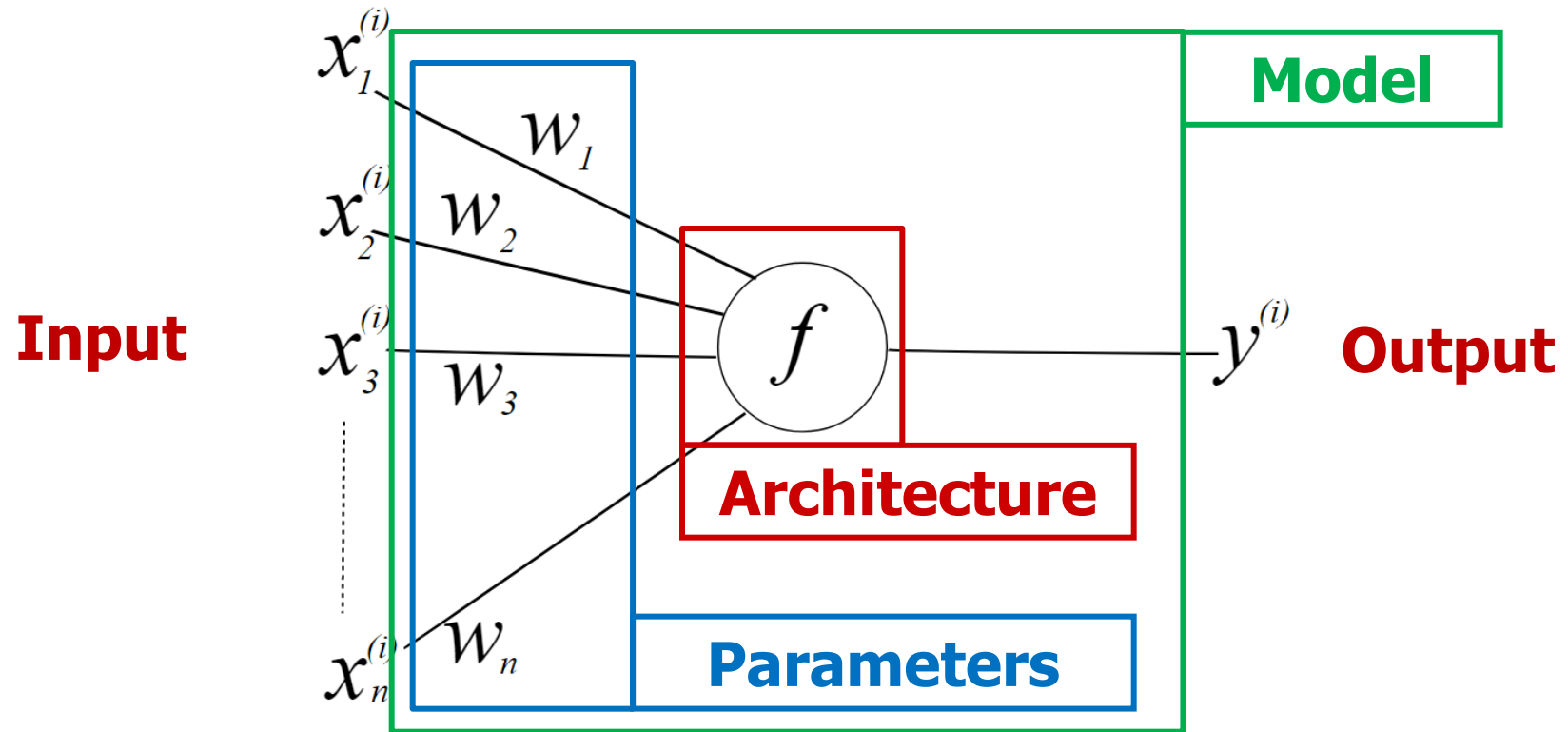
Introduction to AI #3 – DNN Models

Ho Seok AHN

hs.ahn@auckland.ac.nz

1. Neural Network

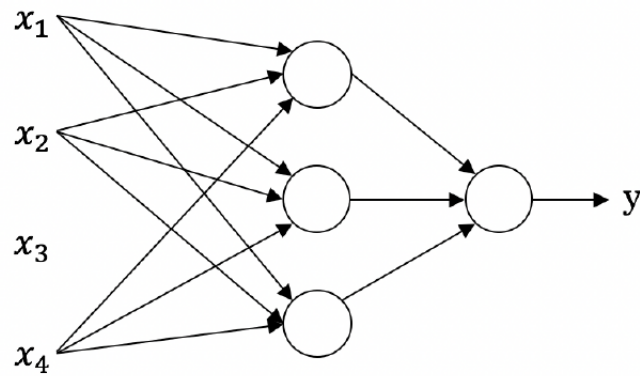
□ What is Model?



Model = Architecture + Parameters

1. Neural Network

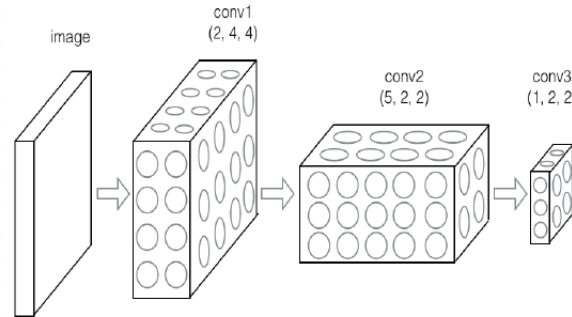
□ Types and models of NN and DNN



Standard NN

Activation function

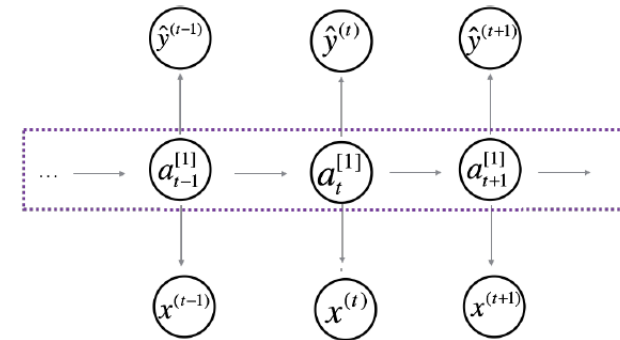
- Sigmoid()
- Tanh()
- ReLU()
- Leaky ReLU()
- ...



Convolutional NN

LeNet-5
AlexNet
VGG
ResNet
Inception
R-CNN

...

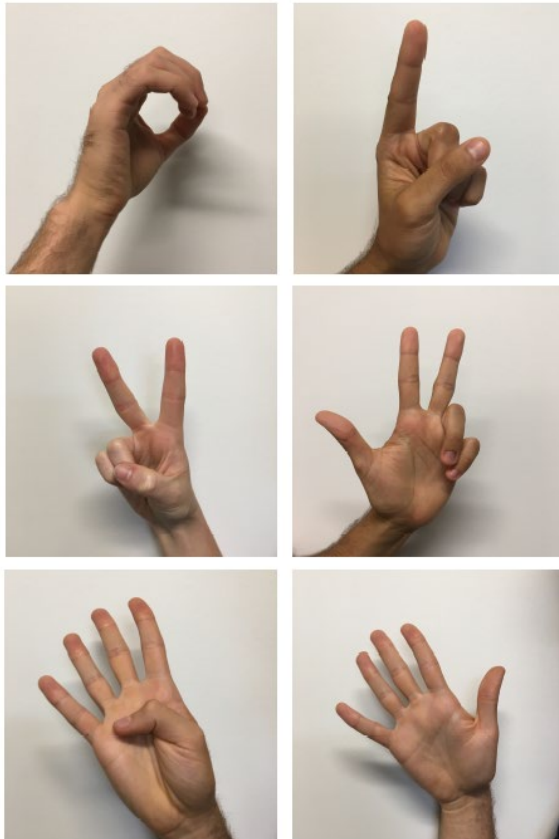


Recurrent NN

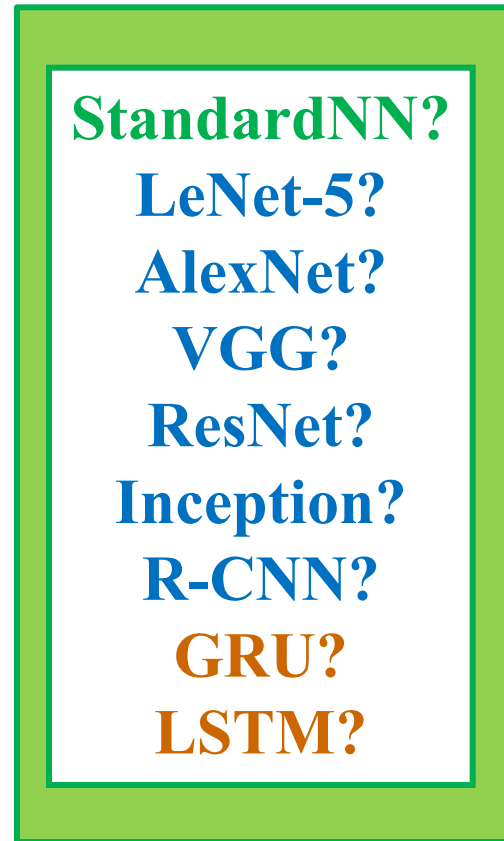
GRU
LSTM

2. Project examples

□ Sign language understanding



Input



0
1
2
3
4
5

Output

2. Project examples

□ Emotion understanding



StandardNN?

LeNet-5?

AlexNet?

VGG?

ResNet?

Inception?

R-CNN?

GRU?

LSTM?



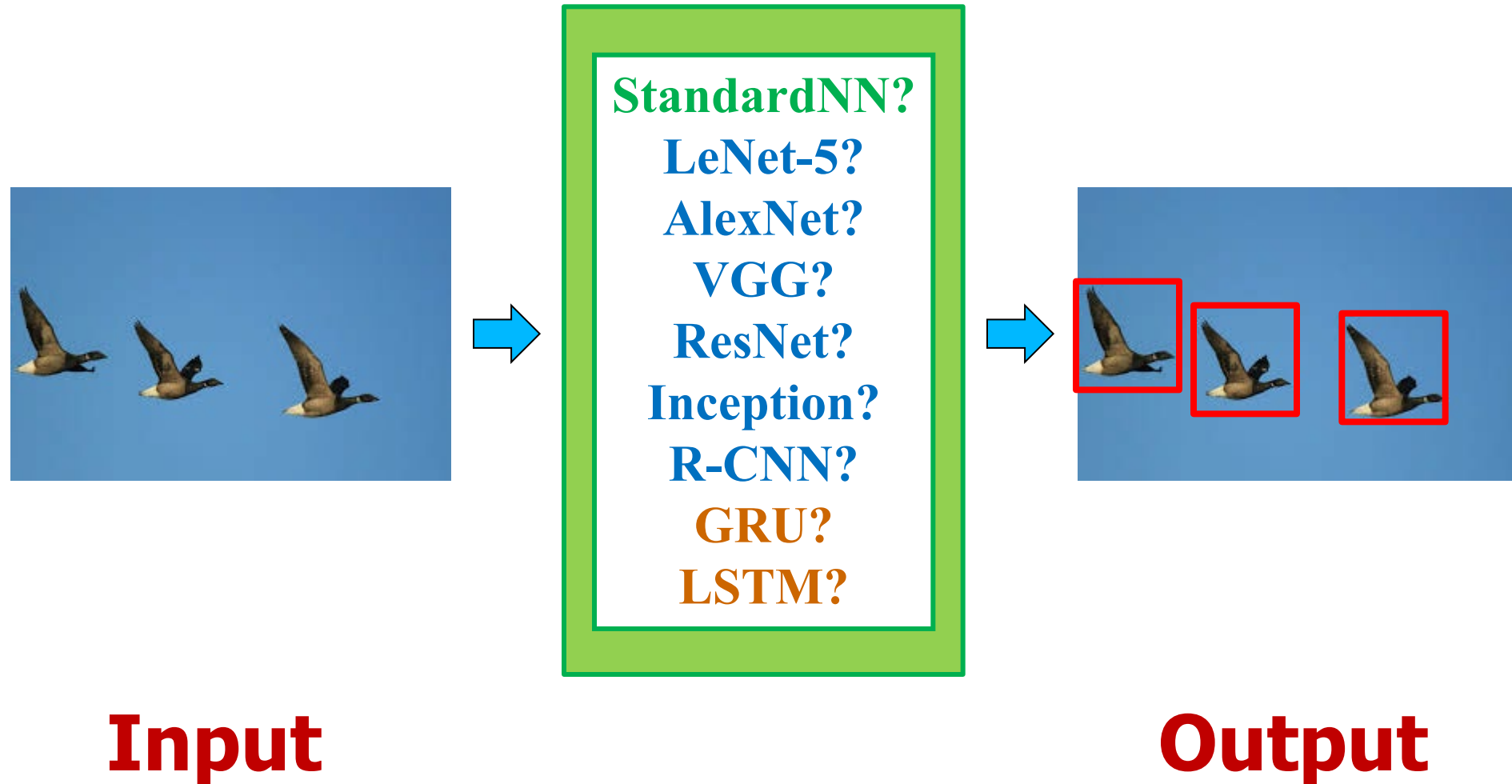
Happy
Sad
Surprise
Angry
...

Input

Output

2. Project examples

□ Object detection



2. Project examples

□ Object recognition



StandardNN?

LeNet-5?

AlexNet?

VGG?

ResNet?

Inception?

R-CNN?

GRU?

LSTM?



Car 0.03

Bike 0.02

Cat 0.75

Dog 0.11

Bird 0.04

... ..

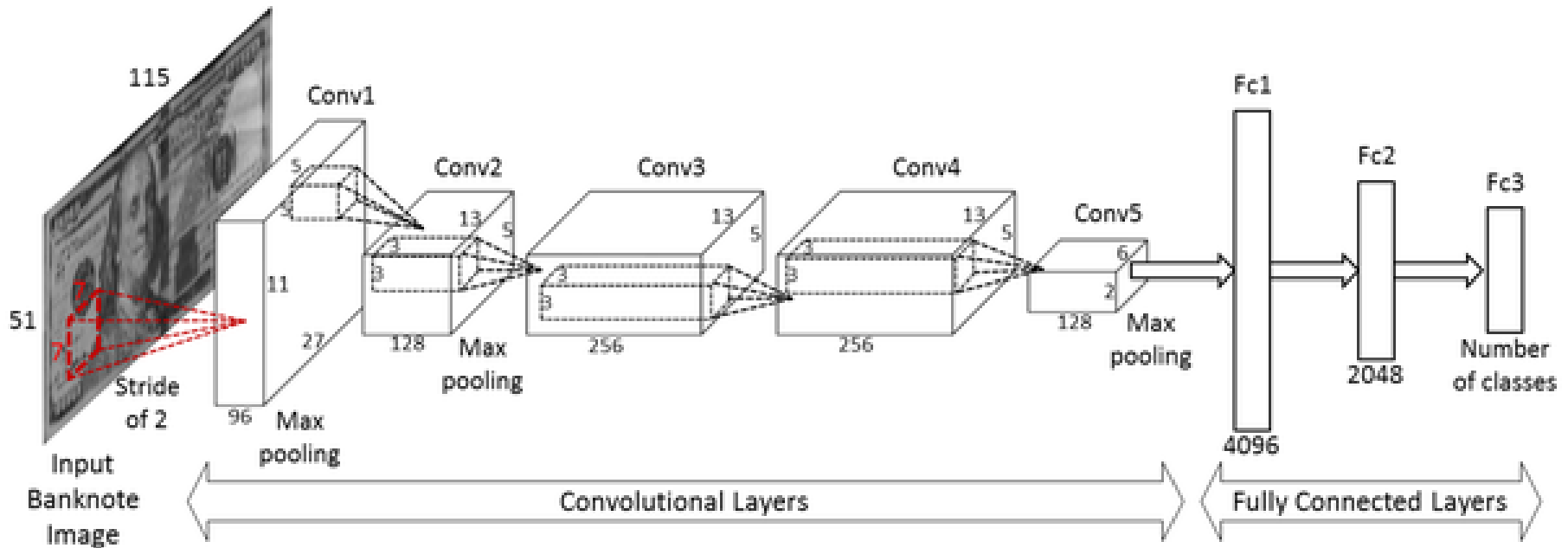
Fish 0.01

Input

Output

3. Different models for CNN

□ CNN



- How deep is **convolutional layers**?
- **Size of kernel matrix** for convolutions?
- Which **polling** methods?
- Which **activation** functions?

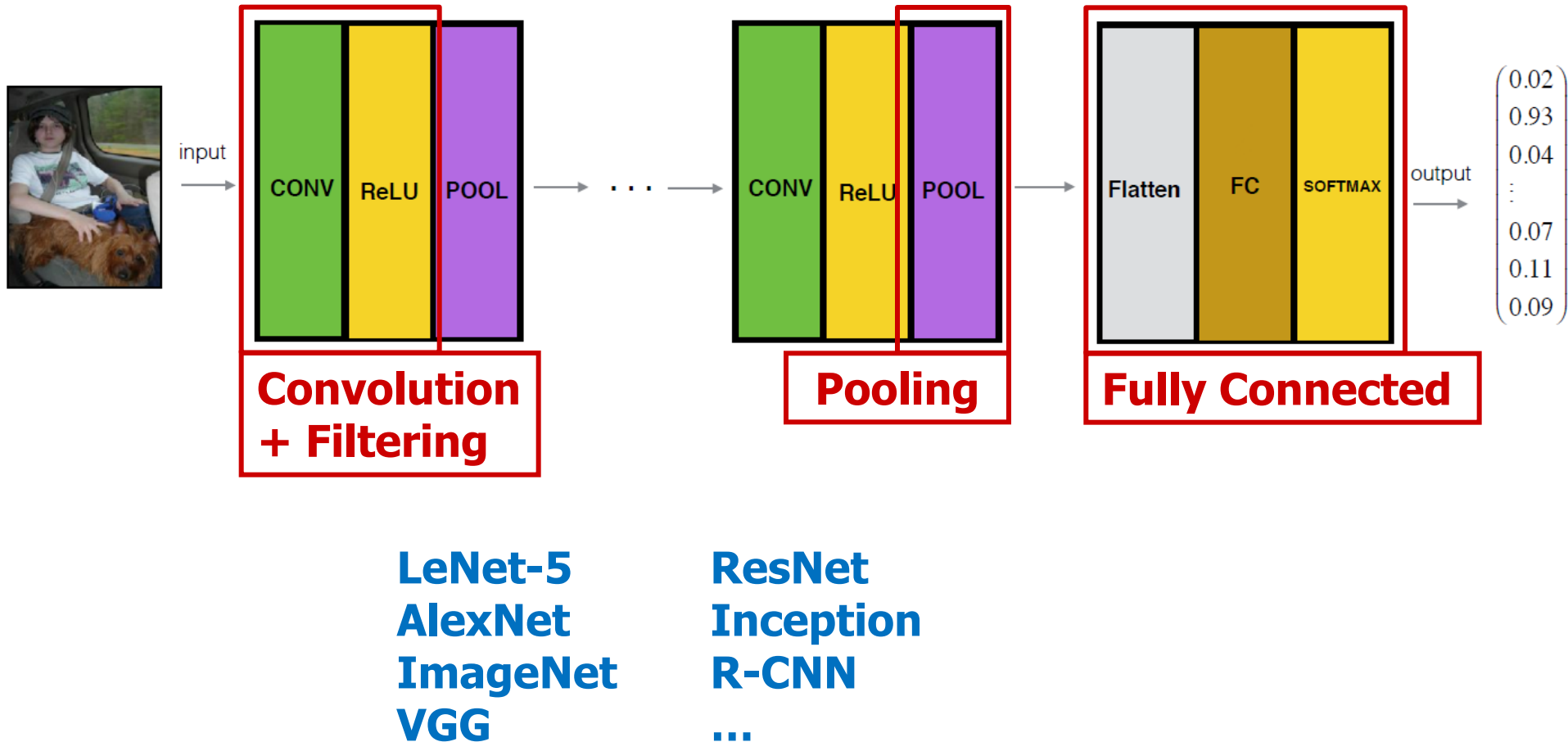
...



**Makes
different
models**

3. Different models for CNN

□ Recap - CNN

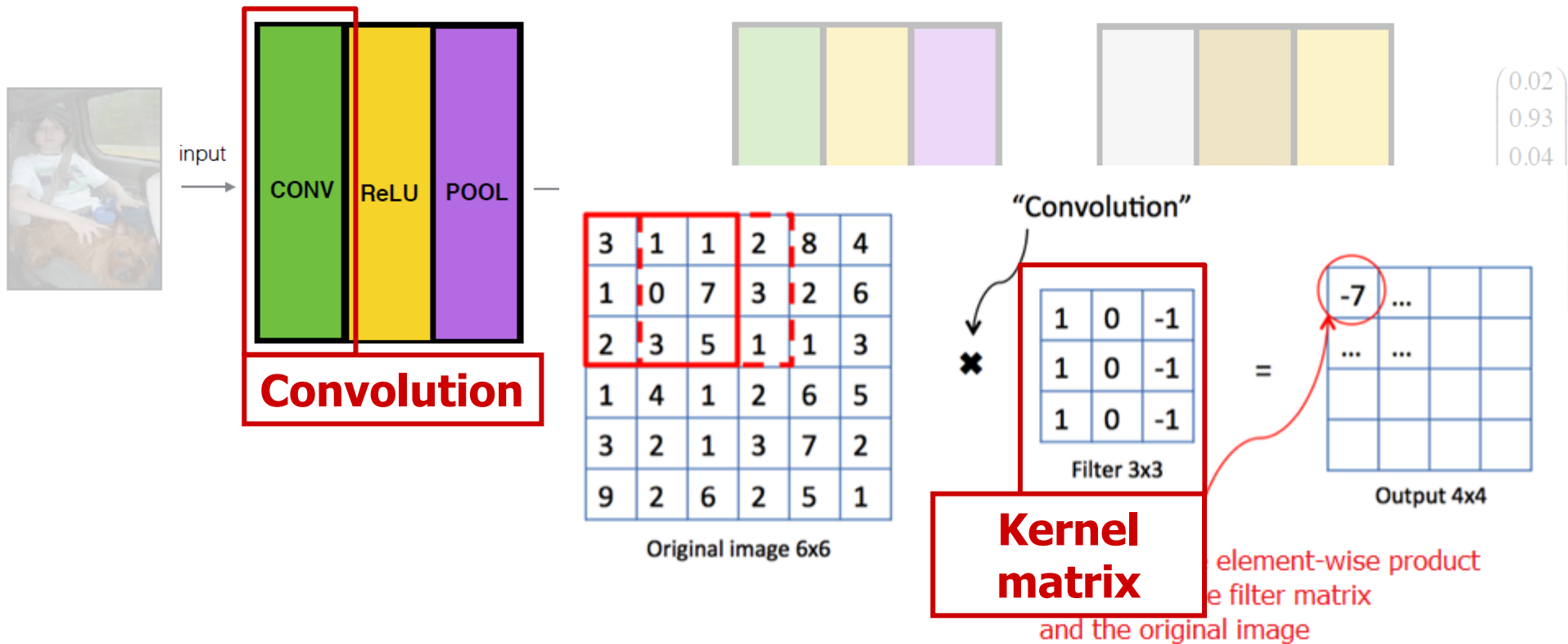


LeNet-5
AlexNet
ImageNet
VGG

ResNet
Inception
R-CNN
...

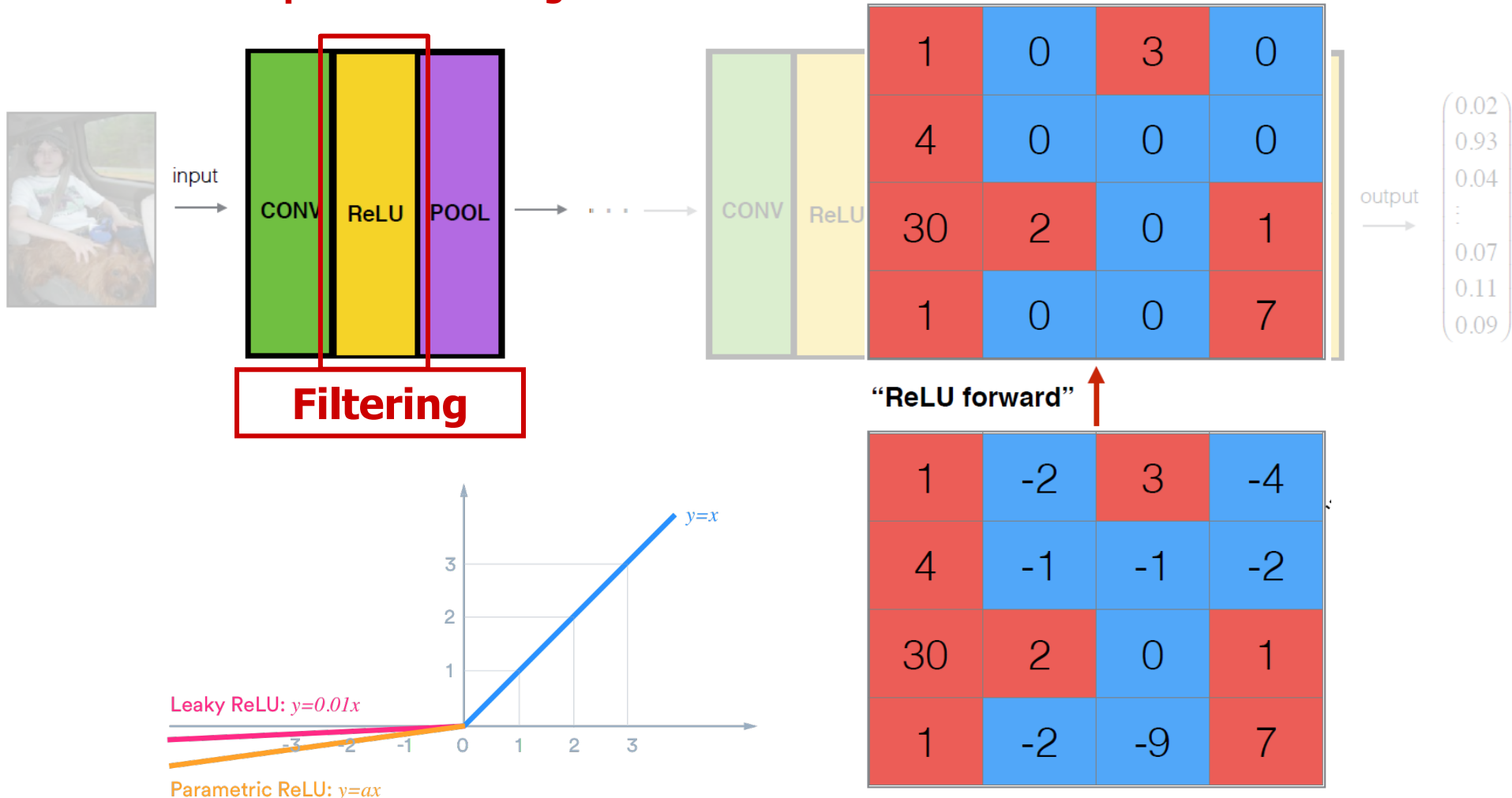
3. Different models for CNN

□ Recap - CNN Convolution



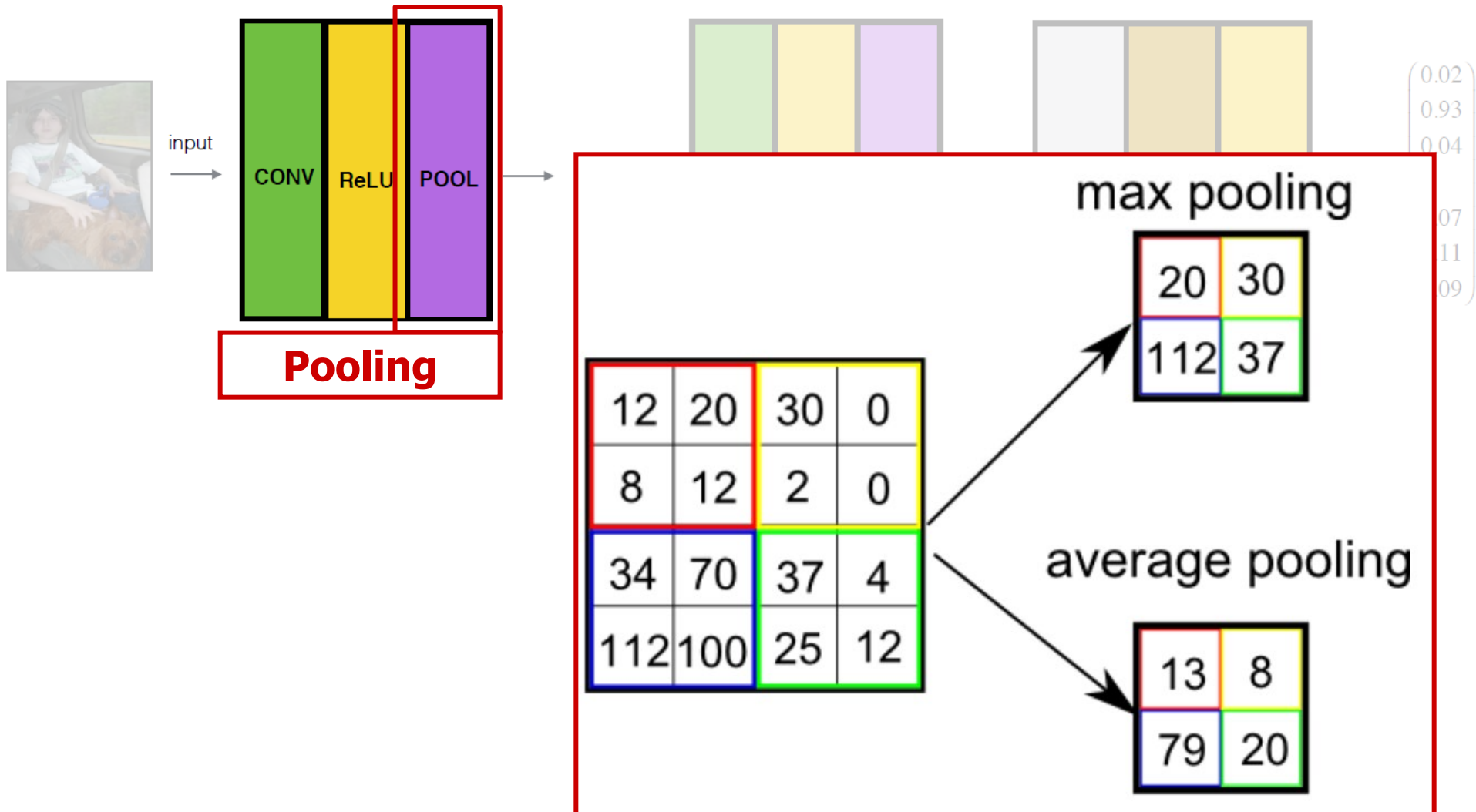
3. Different models for CNN

□ Recap - CNN Filtering



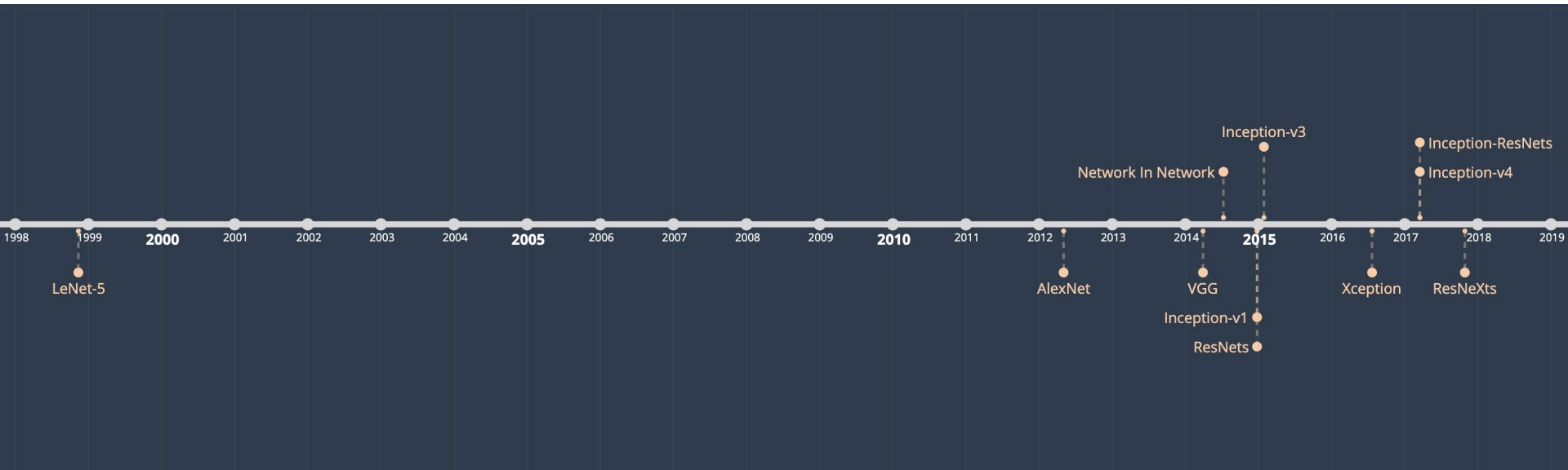
3. Different models for CNN

□ Recap - CNN Polling



3. Different models for CNN

□ CNN



Simple models

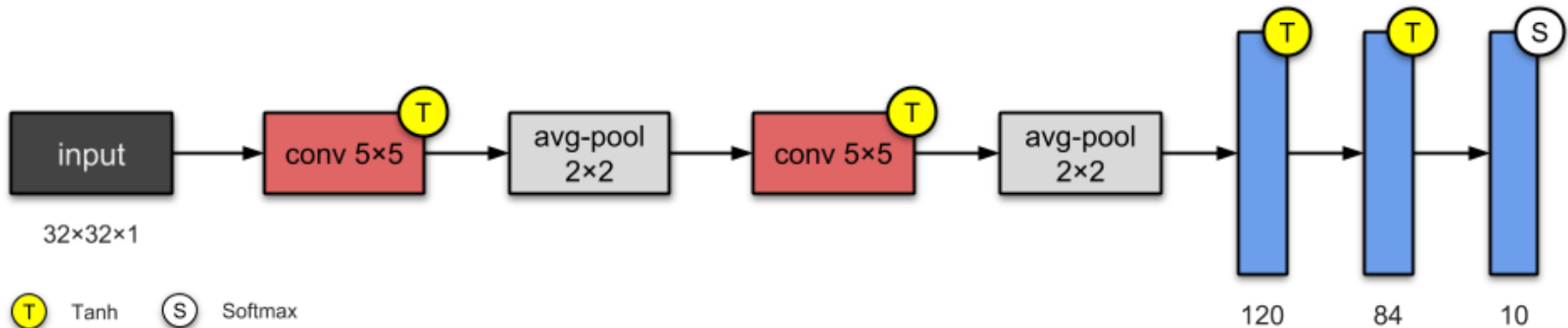


Complex models

3. Different models for CNN

□ LeNet-5 (1998)

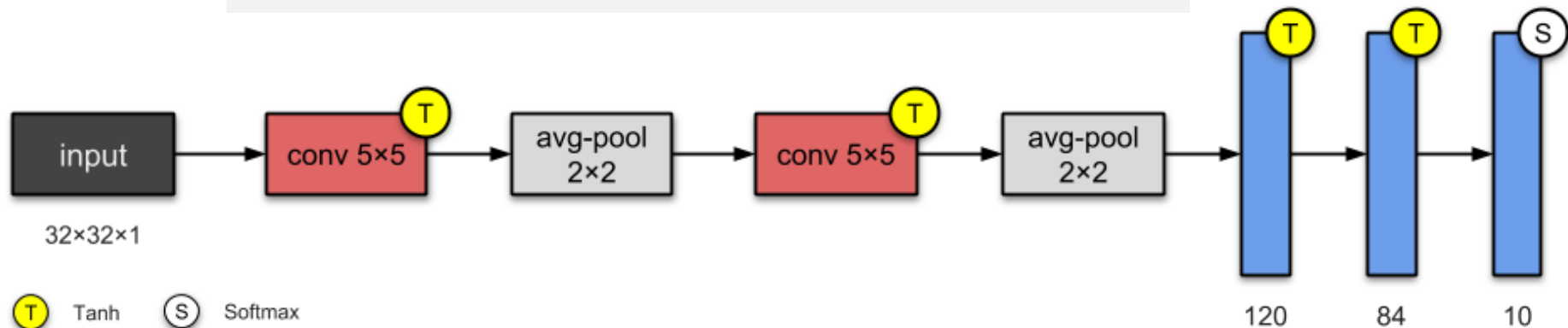
- one of the **simplest** architectures.
 - has 3 convolutional and 2 sub-sampling layers. → named as “5”
 - has about 60K parameters.
 - The average-pooling layer as we know it now was called a “sub-sampling layer”.
- This architecture has become the **standard ‘template’**: **stacking convolutions and pooling layers, and ending the network with one or more fully-connected layers.**



3. Different models for CNN

□ LeNet-5 (1998)

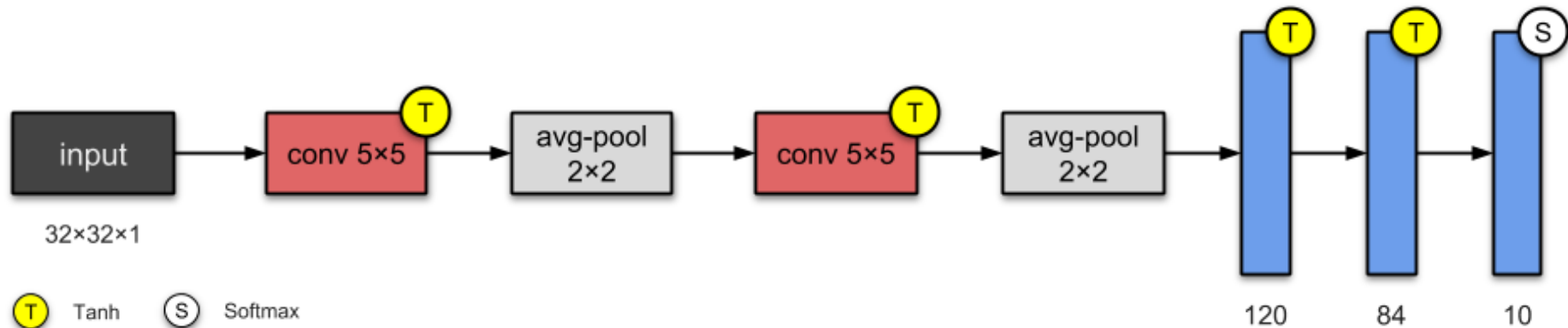
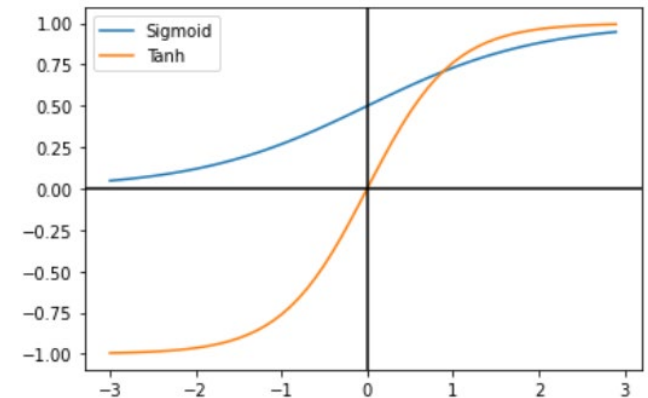
```
model.add(layers.Conv2D(filters=6, kernel_size=(3, 3),  
activation='relu', input_shape=(32,32,1)))  
model.add(layers.AveragePooling2D())  
  
model.add(layers.Conv2D(filters=16, kernel_size=(3, 3),  
activation='relu'))  
model.add(layers.AveragePooling2D())  
  
model.add(layers.Flatten())  
  
model.add(layers.Dense(units=120, activation='relu'))  
  
model.add(layers.Dense(units=84, activation='relu'))  
  
model.add(layers.Dense(units=10, activation = 'softmax'))
```



3. Different models for CNN

□ Why **Hyperbolic Tangent** rather than **Sigmoid**?

- Wider output range (-1~1 vs. 0~1)
- Zero-centered → Stability on learning
- Bigger gradient (1 vs. 0.25) → faster learning

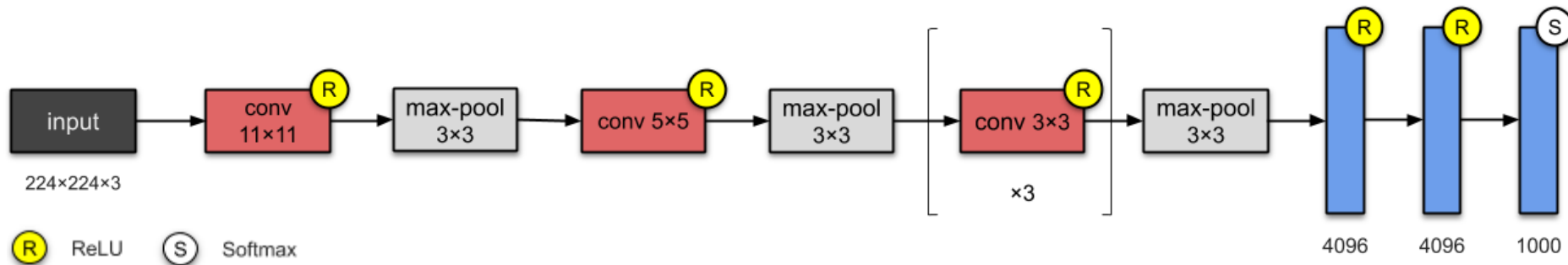


3. Different models for CNN

□ AlexNet (2012)

- invented by University of Toronto, Canada
- **just stacked a few more layers** onto LeNet-5.
- has 8 layers: 5 convolutional and 3 fully-connected layers.
- has about 60M parameters.

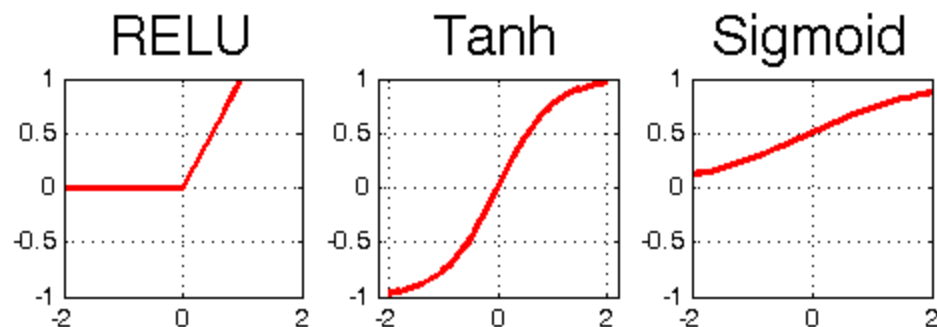
→ This architecture is **the first one to implement Rectified Linear Units (ReLUs)** as activation functions that trains much faster.



3. Different models for CNN

□ Why **ReLU** rather than **Hyperbolic Tangent**?

- **Tanh** has a critical issue; Gradient Vanishing (slow learning when the input is too small or big, when gradient is nearly 0)
- **ReLU** has gradient 1 when the input is over 0 ($x > 0$), so it is more efficient for deep NN.
- **ReLU** requires simple calculation, $\max(0, x)$, but Tanh requires expensive calculation, $\exp()$.
- **ReLU** deactivates some neurons when the input is less than 0, so it learns key features more efficiently.



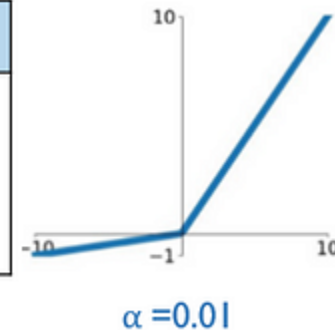
3. Different models for CNN

□ Any issues on ReLU?

□ Dying ReLU

- When $x < 0$, we have dead neurons.
- Can use **different ReLU**, such as **Leaky ReLU**, $f(x) = \max(0.01x, x)$

Function	Equation	Range	Derivative
Leaky ReLu	$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$-\infty, +\infty$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$



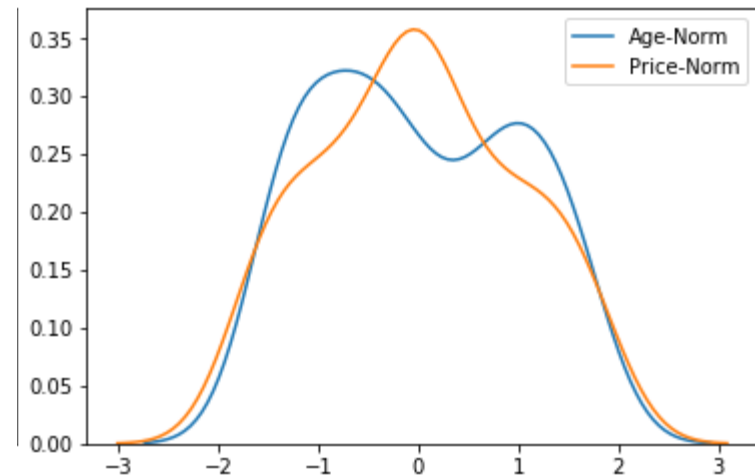
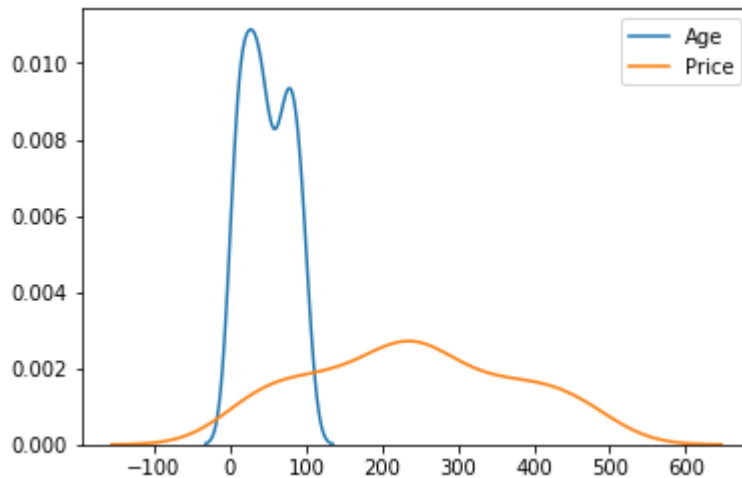
3. Different models for CNN

□ Any issues on ReLU?

□ Not a Zero-centered

➔ Not a big issue.

➔ Can use **Batch Normalization**



3. Different models for CNN

□ Comparisons of activation functions

Function	Range	0-centered	Saturation	Vanishing Gradient	Computation
Sigmoid	0,1	No	For negative and positive values	Yes	Compute-intensive
Tanh	-1,1	Yes	For negative and positive values	Yes	Compute-intensive
ReLu	0, $+\infty$	No	For negative values	Yes (Better than sigmoid and tanh)	Easy to compute
Leaky ReLu	$-\infty, +\infty$	Close	No	No	Easy to compute